1. Servidores ASGI y workers

Una aplicación FastAPI es una aplicación ASGI: no se ejecuta sola, necesita un servidor ASGI como Uvicorn, Hypercorn o Granian. En producción se usa el comando `fastapi run main.py`, que inicia Uvicorn sin recarga automática y escuchando en 0.0.0.0 puerto 8000, o bien `uvicorn main:app --host 0.0.0.0 --port 80` directamente.

Un único proceso de Python ejecuta el código en un solo núcleo de CPU. Para aprovechar varios núcleos se levantan varios procesos worker. Uvicorn lo permite con la opción `--workers 4` (también disponible en `fastapi run --workers 4`). Cada worker es un proceso independiente con su propia memoria: si la aplicación carga un modelo de machine learning de 1 GB, cuatro workers consumen alrededor de 4 GB.

Otra opción clásica es usar Gunicorn como administrador de procesos con workers de Uvicorn: `gunicorn main:app --workers 4 --worker-class uvicorn.workers.UvicornWorker --bind 0.0.0.0:8000`. Gunicorn supervisa los workers y reinicia los que fallan. Como regla práctica, la documentación de Gunicorn sugiere empezar con (2 x cantidad de núcleos) + 1 workers y ajustar con pruebas de carga, ya que el número ideal depende de cuánto tiempo pasa la aplicación esperando E/S.

2. Contenedores con Docker

La forma más común de desplegar FastAPI es empaquetarla en una imagen de Docker. Un Dockerfile típico parte de una imagen oficial liviana de Python, por ejemplo python:3.12-slim, copia primero el archivo requirements.txt e instala las dependencias con `pip install --no-cache-dir -r requirements.txt`, y recién después copia el código de la aplicación. Ese orden aprovecha la caché de capas: si solo cambia el código, no se vuelven a instalar las dependencias.

El comando final se escribe en forma exec (lista JSON) para que el proceso reciba correctamente las señales de apagado: CMD ["fastapi", "run", "app/main.py", "--port", "80"].

Si el contenedor corre en un clúster como Kubernetes, lo recomendable es un solo proceso Uvicorn por contenedor (sin --workers) y escalar aumentando la cantidad de réplicas: el clúster ya se encarga de la replicación, del balanceo de carga y de reiniciar los contenedores caídos. Los múltiples workers dentro de un contenedor tienen más sentido en un único servidor o en despliegues simples con Docker Compose.

3. HTTPS, proxies y buenas prácticas

Normalmente FastAPI no maneja HTTPS por sí misma: un proxy de terminación TLS como Traefik, Nginx, Caddy o el balanceador de carga del proveedor de nube recibe el tráfico cifrado, renueva los certificados (por ejemplo con Let's Encrypt) y reenvía las peticiones en HTTP plano a la aplicación.

Detrás de un proxy hay que indicarle a Uvicorn que confíe en los headers X-Forwarded-For y X-Forwarded-Proto con las opciones `--proxy-headers` y `--forwarded-allow-ips`, para que la aplicación conozca la IP real del cliente y genere URLs con https. Si la API se sirve bajo un prefijo de ruta (por ejemplo /api/v1) que el proxy elimina, se configura `root_path`.

Otras recomendaciones para producción: no usar --reload; desactivar o proteger /docs si la API no es pública; leer secretos desde variables de entorno y nunca desde el código; configurar logs estructurados; exponer un endpoint de salud (por ejemplo /health) para los chequeos del orquestador; y definir límites de memoria y CPU por contenedor.